北京理工大學
BEIJING INSTITUTE OF TECHNOLOGY

第8章 集合

本章知识点

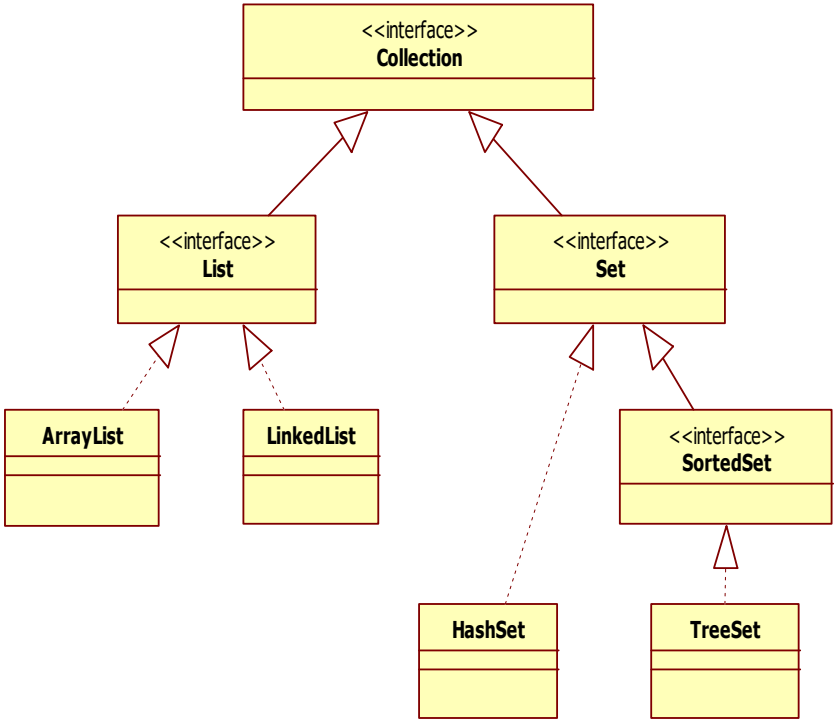
- Java集合框架
- List
  - ArrayList
  - LinkedList
- Set
  - HashSet
  - TreeSet
- Map
  - HashMap
- 泛型和lamda表达式

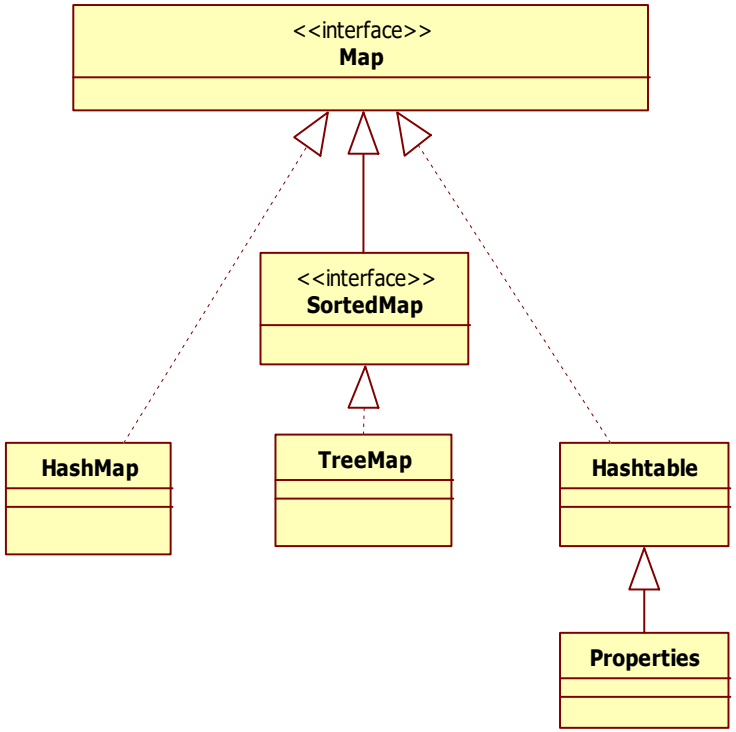
Java中的集合

- Java为程序设计者封装了很多数据结构，它们位于java.util包下，称作Java的集合类，
- 常用的各种数据结构：List表、Set集合、Map映射等。

8.1.1 集合框架的常用部分

- Java集合框架的根是两个接口：Collection和Map。





8.1.1 集合框架的常用部分

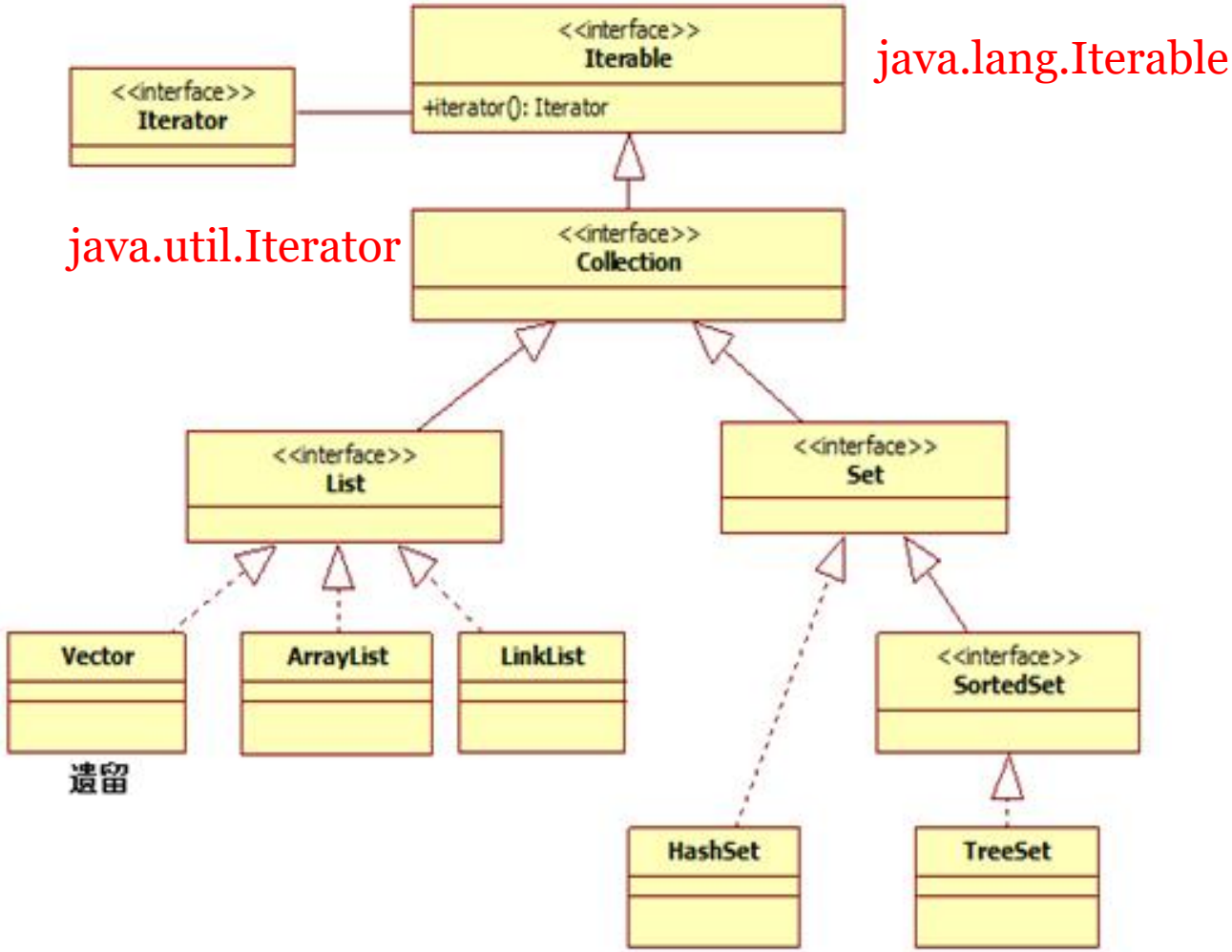
- 根据Collection和Map的架构，Java中的集合可以分为三大类
  - List：有序集合，它会记录每次添加元素的顺序，元素可以重复；
  - Set：无序集合（不准确），它不记录元素添加的顺序，元素不可重复；
  - Map：键值对的集合，它的每个元素都由键值key和取值value对应组成，键值和取值分别存储，键值是无序集合Set，取值是有序集合List。

- 集合与数组有一个明显的不同，就是在Java的集合中只能存储对象，而不能存储基本类型的数据。

- 接口中的iterator()方法
  - “遍历”：按照某种次序将集合中的元素全部访问到，且每个元素只访问一次。
  - Iterator iterator()方法：用于获取迭代器，从而遍历Collection中的所有元素。

8.1.2 迭代器Iterator接口

- Iterator接口，作为Java集合框架的成员，它的作用是遍历集合中的元素，而不必知道该序列的底层结构。



8.1.2 迭代器Iterator接口

北京理工大学 BEIJING INSTITUTE OF TECHNOLOGY

• 集合获取迭代器：Iterator iterator()方法。

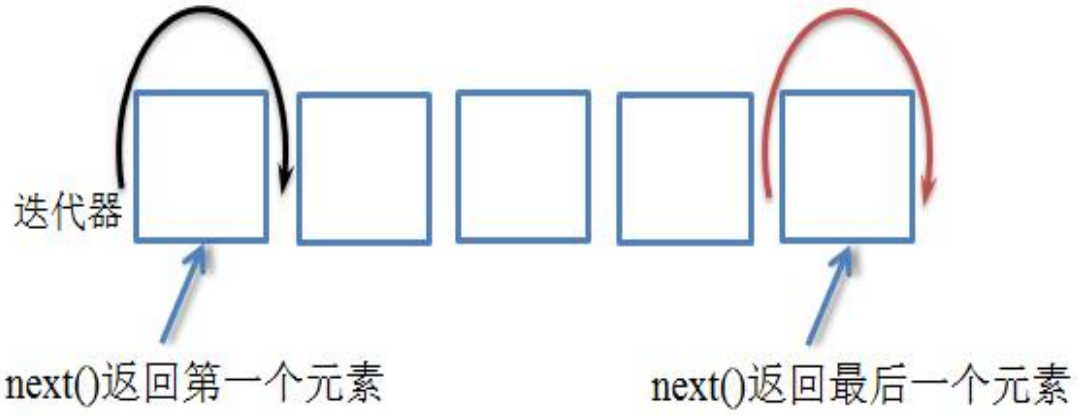
Iterator中的方法

default void	**forEachRemaining(Consumer<? super E> action)** Performs the given action for each remaining element until all elements have been processed or the action throws an exception.
boolean	**hasNext()** Returns true if the iteration has more elements.
E	**next()** Returns the next element in the iteration.
default void	**remove()** Removes from the underlying collection the last element returned by this iterator (optional operation).

8.1.2 迭代器Iterator接口

（1）next()

- Object next()。迭代器会记录迭代位置的变更，它使用next()方法将迭代位置向下一个元素移动，并返回刚刚越过的那个元素。



8.1.2 迭代器Iterator接口

北京理工大学 BEIJING INSTITUTE OF TECHNOLOGY

（2）hasNext()

- boolean hasNext()：如果集合中还有未被遍历的元素，返回true，即迭代器还可以继续向后移动的时候返回true。

- 遍历时，在hasNext()方法的控制下，通过反复调用next()方法，逐个访问集合中的各个元素。

8.1.2 迭代器Iterator接口

北京理工大学 BEIJING INSTITUTE OF TECHNOLOGY

```
Collection c = new ArrayList();  //创建一个ArrayList对象

c.add("Java");
c.add("Struts");
c.add("Spring");
```

```
Iterator it = c.iterator();   //(1)iterator()方法获取迭代器
while (it.hasNext()){  //(2)hasNext()方法控制迭代
        //(3)next()方法获取迭代元素
        String element = (String)it.next();
        System.out.println(element);
}
```

Sample:CrossContainerIteration

（3）remove()

- void remove()：删除集合中上一次调用next()方法时返回的元素。

- 使用remove()删除某位置的元素时，必须先调用next()方法使迭代器跳过该元素。

设集合中已存在若干元素，删除第一个元素的方法：

```
Iterator it = c.iterator();
it.next();     //越过第一个元素
it.remove();  //删除第一个元素
```

Example :IteratorDemo.java

传统的for-each循环遍历


```
Collection c = new ArrayList();  //创建一个ArrayList对象

c.add("Java");
c.add("Struts");
c.add("Spring");
```

循环变量 : 集合对象

```
for(Object element: c){
        String str = (String)element;
        System.out.println(str);
}
```

自动转化为使用迭代器循环

foreach+lambda表达式遍历

北京理工大学 BEIJING INSTITUTE OF TECHNOLOGY

- Java SE 8开始，Iterable接口中新增了forEach方法

Modifier and Type	Method and Description
default void	forEach(Consumer<? super T> action) Performs the given action for each element of the Iterable until all elements have been processed or the action throws an exception.
Iterator<T>	iterator() Returns an iterator over elements of type T.

- 参数action：lambda表达式，描述遍历的行为
- lambda表达式能够将行为做为方法的参数

- **需要函数式接口的支持**

@FunctionalInterface 函数式接口

函数式接口有且只有一个abstract方法。但可以有多个static 方法和default 方法。

如果接口用@FunctionalInterface 修饰，并包括多个抽象方法，则编译时会报错。

- lambda表达式的本质是函数式接口的匿名内部类
- 只保留函数式接口抽象方法的参数列表和方法体，并在参数列表和方法体间用箭头隔开。

函数式接口

```
ReturnNoParam return = () ->
System.out.print("ReturnNoParam");
        return 1;
    };
```

LambdaExample.java

LambdaExampleSimple.java

lambda表达式


```
ReturnNoParam return = () ->
System.out.print("ReturnNoParam");
        return 1;
    };
```

等价于

```
ReturnNoParam  return;
return=new ReturnNoParam(){
        public void method(){
                System.out.println("NoReturnNoParam");
                  return 1;
         };
                  };
```

```
@FunctionalInterface//函数式接口
public interface ReturnMultiParam {
        int method(int a, int b);
}
```

```
//多个参数有返回值的Lamda表达式
    ReturnMultiParam returnMultiParam = (int a, int b) -> {
        System.out.println("ReturnMultiParam param:" + "{" + a +
"," + b +"}");
        return 1;
    };
```

lambda表达式的简化

- 不需要声明参数类型，编译器可以统一识别参数值
- 一个参数无需定义圆括号
- 如果方法体只包含了一条语句，就不需要使用大括号
- 如果方法体只包含了一条语句且为**return**语句除了不需要使用大括号还可以省略**return**关键字。

LambdaExampleSimple.java

lambda表达式中的方法引用和构造器引用

- 只有当方法体只有一条代码时才可以使用，其本质是表达式的进一步简化，省略形参列表和箭头

（1）引用类方法

（参数列表）->某个类.某个方法（参数列表）；　某个类：：某个方法

（2）引用实例的方法

（参数1，其他参数）->参数1.某个方法（其他参数）；

参数1类型：：某个方法

（3）引用特定对象的方法

（参数列表）->某个对象.某个方法（参数列表）；

某个对象：：某个方法

（4）构造器引用

类名：：new

LambdaReference.java　LambdaReference2.java

java提供的函数式接口

函数式接口	参数类型	返回类型	用途
Consumer<T> 消费型接口	T	void	对类型为T的对象应用操作，包含方法：void accept(T t)
Supplier<T> 供给型接口	无	T	返回类型为T的对象，包含方法：T get();
Function<T, R> 函数型接口	T	R	对类型为T的对象应用操作，并返回结果。结果是R类型的对象。包含方法：R apply(T t);
Predicate<T> 断定型接口	T	boolean	确定类型为T的对象是否满足某约束，并返回boolean 值。包含方法boolean test(T t);

LambdaInterface.java

www.atguigu.com

其他接口

函数式接口	参数类型	返回类型	用途
BiFunction<T, U, R>	T, U	R	对类型为 T, U 参数应用操作，返回 R 类型的结果。包含方法为 R apply(T t, U u);
UnaryOperator<T> (Function子接口)	T	T	对类型为T的对象进行一元运算，并返回T类型的结果。包含方法为 T apply(T t);
BinaryOperator<T> (BiFunction 子接口)	T, T	T	对类型为T的对象进行二元运算，并返回T类型的结果。包含方法为 T apply(T t1, T t2);
BiConsumer<T, U>	T, U	void	对类型为T, U 参数应用操作。包含方法为 void accept(T t, U u)
ToIntFunction<T> ToLongFunction<T> ToDoubleFunction<T>	T	int long double	分别计算int、long、double、值的函数
IntFunction<R> LongFunction<R> DoubleFunction<R>	int long double	R	参数分别为int、long、double 类型的函数

for-each+lambda表达式遍历

北京理工大学 BEIJING INSTITUTE OF TECHNOLOGY

- Iterable接口中的forEach方法

Modifier and Type	Method and Description
default void	**forEach**(Consumer<? super T> action) Performs the given action for each element of the Iterable until all elements have been processed or the action throws an exception.
Iterator<T>	**iterator**() Returns an iterator over elements of type T.

- 参数action：lambda表达式，描述遍历的行为

foreach+lambda表达式遍历


```
Collection c = new ArrayList();  //创建一个ArrayList对象

c.add("Java");
c.add("Struts");
c.add("Spring");
```

```
c.forEach( ele -> System.out.println(ele) );
```

集合对象　　循环变量　　->

```
c.forEach( (ele) -> {System.out.println(ele);} );
```

ForeachExample.java

泛型的本质是参数化类型，即所操作的数据类型作为参数

主要包括泛型类 泛型接口 泛型方法

搭配通用符使用

泛型初始化过程中，一旦给定了参数类型之后，参数类型就会被限制

（1）泛型类声明

型类的声明格式如下：

class 类名<泛型列表>

例如：

class A<E>{...}

其中，A是泛型类的名称，E是其中的泛型列表中一个类型参数，它可以是任何类或接口，但不能是基本数据类型。泛型声明时，E可以作为类的成员变量的类型、方法的返回类型以及局部变量的类型。泛型列表也可以有多个,必须用逗号隔开，格式如下：

class A<E,F>{...}

泛型类的结构和普通类完全类似，由成员变量和方法构成。例如：

```
class Pair<T>{
        private T first;
        private T second;
        public Pair(){first=null; second=null;}
        public Pair(T first,T second){this.first=first;
this.second=second;}
        public T getFirst(){return first;}
        public T getSecond(){return second;}
        public void setFirst(T newValue){first=newValue;}
        public void setSecond(T newValue)
{second=newValue;}
 }
```

（2）使用泛型类声明对象

与普通类不同的是，使用泛型类型声明和创建对象时，类名后多了一对“< >”符号。而且要用具体的类型替换泛型类中的泛型。如：

Pair<String> pair1;//声明一个泛型对象

pair1= new Pair<String>("Hello","Java");//创建泛型对象

泛型类的应用。

```
public class PairTest{
  public static void main(String[] args){
    Pair<String> pair=new Pair<String>("Hello","Java");
            System.out.println("first="+pair.getFirst());
            System.out.println("second="+pair.getSecond());
  }
}
```

Example：PairTest.java
（cn.yhn.generic）

可以定义一个带有参数类型的方法即泛型方法。

泛型方法所在的类可以是泛型类，也可以不是泛型类。

创建一个泛型方法常用的形式如下：

[访问修饰符] [static] [final] <泛型列表> 返回值 方法名([形式参数列表])

```
class GenericMethod{
 public <T> void f(T x){
        System.out.println(x.getClass().getName());
 }
}
public class GenericMethodTest{
        public static void main(String[] args){
                GenericMethod gm=new GenericMethod();
                gm.f(" ");
                gm.f(1);
                gm.f(1.0f);
                gm.f('c');
                gm.f(gm);
        }
}
```

泛型接口的定义与泛型类非常相似，它的声明形式如下：

interface 接口名<泛型列表>

```
interface MinMax<T >{
        T min();
        T max();
}
```

8.2.3泛型接口

北京理工大学 BEIJING INSTITUTE OF TECHNOLOGY

定义一个类**MyClass**来实现这个接口，代码如下：

```
class MyClass<T extends Comparable<T>> implements MinMax<T>{
        T[] vals;
        MyClass(T[] ob){
                vals = ob;
        }
        public T min(){
                T val = vals[0];
                for(int i=1; i<vals.length; ++i)
                        if(vals[i].compareTo(val)<0)val=vals[i];
                return val;
        }
        public T max(){
                T val = vals[0];
                for(int i=1;i<vals.length;++i)
                        if (vals[i].compareTo(val)>0)val=vals[i];
                return val;
        }
}
```

泛型接口的应用：**MyClassTest.java**

使用extends关键字指定泛型实例化参数只能是指定类或指定类的子类，super关键字可以指定泛型实例化时的参数只能是指定类或指定类的父类。

“<? extends E>”中“？”表示通配，“<? extends E>”是“上界通配符”，表示该类型的上限是E。“<? super E>”是“下界通配符”，表示该类型的下限是E。

ConvariantArrays.java

8.2.5 使用泛型

在泛型出现之前，一旦将一个对象放在Java的集合中，集合就会忘记对象的类型，所有对象都被当做Object处理。
当从集合中取出对象后，就需要进行强制类型转换。
使用泛型后，不需要强制转换。

```
Collection<String> c = new ArrayList<String>();

Iterator<String> it = c.iterator();
```

- ArrayList类的API文档

boolean	**add**(E e) Appends the specified element to the end of this list.
void	**add**(int index, E element) Inserts the specified element at the specified position in this list.

- 【说明】从Java SE 8开始，等号右侧的“<>”中的泛型可以不写，类型进行自动推断。

```
Collection<String> c = new ArrayList<>();

Iterator<String> it = c.iterator();
while (it.hasNext(){
        String element = it.next();    //不需要强转
        System.out.println(element);
}
```

```java
class Parent{
}
class Children extends Parent{
}
public class GenericExtendsDemo {

    public static void main(String[] args) {
        Collection<Parent> c = new ArrayList<>();

        Collection<Children> c2 = new ArrayList<>();
        c2.add(new Children());
        c2.add(new Children());

        c.add(new Parent());   //E类型
        c.addAll(c2);    //E的子类类型
        c.add(new String());     //无关类型
    }
}
```

- 线性表结构
  - 顺序表
  - 链表

8.3.1 List接口

- List接口添加的**面向位置**的方法
  - void add(**int index**, Object element)：在指定位置index上插入元素element。
  - boolean addAll(**int index**, Collection c)：指定位置index上插入集合c中的所有元素，如果List对象发生变化返回true。
  - Object get(**int index**)：返回指定位置index上的元素。
  - Object remove(**int index**)：删除指定位置index上的元素。
  - Object set(**int index**, Object element)：用元素element取代位置index上的元素，并且返回旧元素的取值。
  - public **int** indexOf(Object obj)：从列表的头部开始向后搜索元素obj，返回第一个出现元素obj的位置，否则返回-1。
  - public **int** lastIndexOf(Object obj)：从列表的尾部开始向前搜索元素obj，返回第一个出现元素obj的位置，否则返回-1。

【例8-2】示范List的常规用法。

- 向List集合添加几个字符串，按索引位置进行插入、修改、删除、查找等操作，并打印集合（根据索引位置遍历集合中的元素）。

Example:ListDemo.java

- **1. ArrayList的底层**
  - ArrayList是List的实现类，它封装了一个可以动态再分配的Object[]数组。
  - ArrayList中的常用方法包括：

    （1）ArrayList()：从Java SE 8 开始，ArrayList()初始化时得到一个空数组，在第一次add()运算时将数组扩容为10。

    （2）ArrayList(int initialCapacity)：创建ArrayList 对象，使用参数initialCapacity设置Object[]数组的长度。

    （3）ensureCapacity():如果要向ArrayList中添加大量元素，可使用此方法对Object[]数组进行指定的扩容。

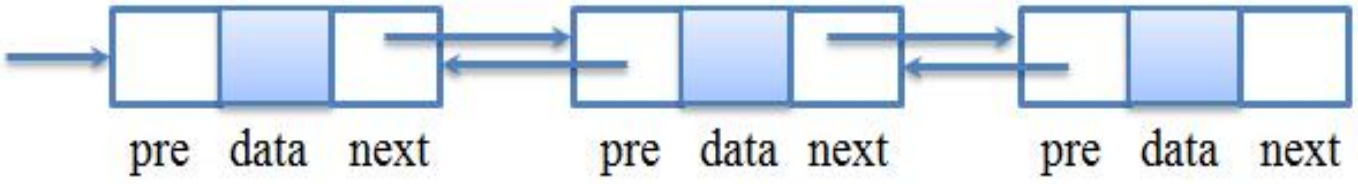
2. 关于remove()方法

- Collection接口中的remove()方法
  - boolean **remove**(Object obj)
  - boolean **removeAll**(Collection c)。

- 使用迭代器循环迭代的过程中，是不能用这些remove()方法删除集合元素的，因为Collection和List中定义的remove()方法对迭代器的修改，与迭代器本身的管理不能同步，从而会引发ConcurrentModificationException异常。
- RemoveDemo.java

8.3.3 LinkedList

- 链表：采用“按需分配”的原则为每个对象分配独立的存储空间（称为结点），并在每个结点中存放序列中下一个结点的引用。
- Java中的链表是双链表，每个结点还存放它前面结点的引用。



选择ArrayList还是LinkedList？？

（1）如果经常要存取List集合中的元素，那么使用ArrayList采用随机访问的形式（get(index)，set(index ,Object element)）性能更好。

（2）如果要经常执行插入、删除操作，或者改变List集合的大小，则应该使用LinkedList。

8.4 Set及其实现类

北京理工大学 BEIJING INSTITUTE OF TECHNOLOGY

- Set按照无序（不太准确）、不允许重复的方式存放对象
- 实现类
  - HashSet：基于散列结构
  - TreeSet：基于查找树结构

8.4.1 Set接口

- Set接口继承自Collection 接口，它没有引入新方法，所以Set就是一个Collection，只是行为方式不同。
- Set不允许集合中存在重复项，如果试图将两个相同的元素加入同一个集合，则添加无效，add()方法返回false。

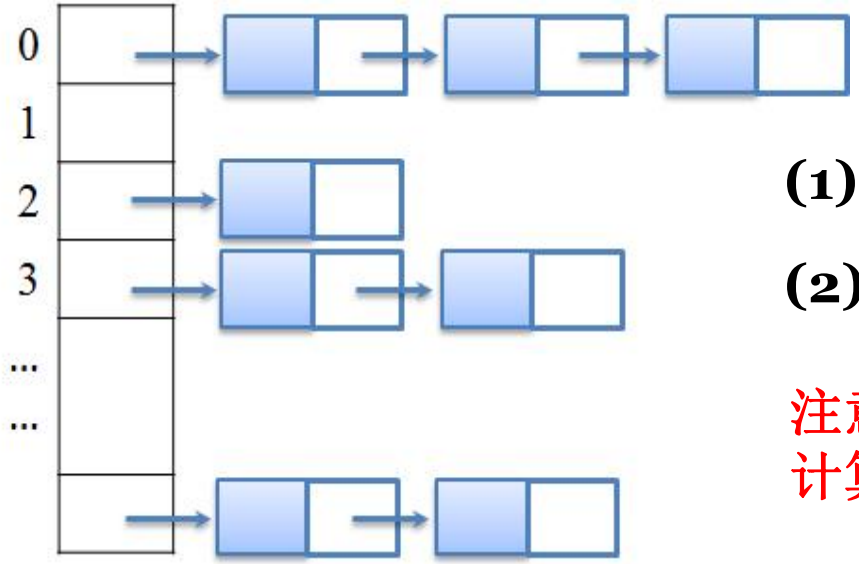
8.4.1 Set接口

- Set依赖数据元素的equals()方法检查对象的唯一性
  - 只要两个对象使用equals()方法比较的结果为true，Set就会拒绝后一个对象的加入（哪怕它们实际上是不同的对象）；
  - 只要两个对象使用equals()方法比较的结果为false，Set就会接纳后一个对象（哪怕它们实际上是相同的对象）。

- 使用Set存放对象时，重写该对象所在类的equals()方法、制定正确的比较规则。

HashSet的底层结构

- HashSet是Set的实现类，它基于可以实现快速查找的散列表（Hash Table）结构。
- 散列表采用按照数据的取值计算数据存储地址的策略，实现对象的“定位”存放，相应也提高了查找效率。



(1) 对象定位-- hashCode()方法

(2) 对象查重-- equals()方法

注意：重写两个方法，保证二者的计算依据相同

2. 散列码和hashCode()方法

- 要将对象存储在基于散列结构的HashSet，自定义类必须按规则重写Object中的hashCode()方法。
- 所谓的规则就是要保证hashCode()方法与重写的equals()方法完全兼容，即如果a.equals(b)为true，那么a和b也必须通过hashCode()方法得到相同的散列码。

3. 向HashSet中添加元素

- HashSet中不允许存在重复的元素，因此add()方法首先尝试查找要添加的对象，只有在该对象不存在的情况下才执行添加。

（1）依据自定义类的hashCode()方法计算得到对象obj的散列码，它是一个整数。（需要自定义类重写hashCode()方法）

（2）将散列码对表长求余，得到对象在散列表中的存储位置p。

（3）如果p位置不发生冲突，则将对象obj插入在p位置的链表中。

3. 向HashSet中添加元素

（4）如果p位置发生冲突，在p位置对应的链表中利用equals()方法查找是否已存在obj对象。（需要自定义类重写equals()方法，规则要与hashCode()方法互相匹配）

- 1）如果某个equals()比较的结果为true，说明obj对象已存在，将其舍弃。
- 2）如果与链表中所有对象的equals()比较的结果均为false，说明obj对象尚未存在，obj插入该链表。

【例8-4】定义一个Student类（具有name和age两个属性），向HashSet集合中添加几个Student对象，并打印该集合。

Example :HashSetDemo.java

1. TreeSet的底层结构

- TreeSet如其名字一样，是一种基于树的集合。TreeSet是Set接口的实现类，秉承了Set不记录对象在集合中出现顺序的特点。
- 是有序的，红黑树结构
- 对象进入TreeSet需要进行一系列比较才能找到正确的位置插入。

2. TreeSet中对象的比较方法：自然比较法和比较器法

- Java提供了两种比较方式：自然比较法和比较器法
- （1）**the *natural comparison method*** 自然比较法----

- TreeSet存储对象所在类要实现Comparable接口，在compareTo()方法中定义对象比较的规则。
  - implement the **java.lang.Comparable** interface (compareTo( )). 内部比较器

  - 有一部分类已实现了java.lang.Comparable接口，如基本类型的包装类、String类等，它们在compareTo()方法中定义好了比较对象的规则。像这样的对象可以直接插入TreeSet集合.

```
public class Student implements Comparable{
        private String name;
        private int age;
        public int compareTo(Object obj) {
                if(obj instanceof Student){
                        Student stu = (Student)obj;
                        return this.name.compareTo(stu.name);//当前对象
-传入对象
                }
                return 0;
        }
}
```

（2） Create a separate class that implements the **Comparator** interface. (compare( ) and equals( ))比较器法

【例8-7】为Student类定义两个比较器ComparatorName和ComparatorNameAge。ComparatorName按name排序，ComparatorNameAge在name相同时继续按age排序。

3. 向TreeSet注入比较器

```
public class ComparatorName implements Comparator{ //按name进行排序
        public int compare(Object obj1, Object obj2) {
                if(obj1 instanceof Student && obj2 instanceof Student){
                        Student s1 = (Student)obj1;
                        Student s2 = (Student)obj2;
                        //按name进行比较
                        return s1.getName().compareTo(s2.getName());
                }
                return 0;
        }
}
```

3. 向TreeSet注入比较器

```
public class ComparatorNameAge  implements Comparator{ //按name和age进行排序
        public int compare(Object obj1, Object obj2) {
                if(obj1 instanceof Student && obj2 instanceof Student){
                        Student s1 = (Student)obj1;
                        Student s2 = (Student)obj2;
                        if(s1.getName().equals(s2.getName())){
                                return s1.getAge()-s2.getAge();
                                }else{
                                return s1.getName().compareTo(s2.getName());
                                }
                }
                return 0;
        }
}
```

3. 向TreeSet注入比较器

```
Set<Student> students = new TreeSet<>(new ComparatorName())
```

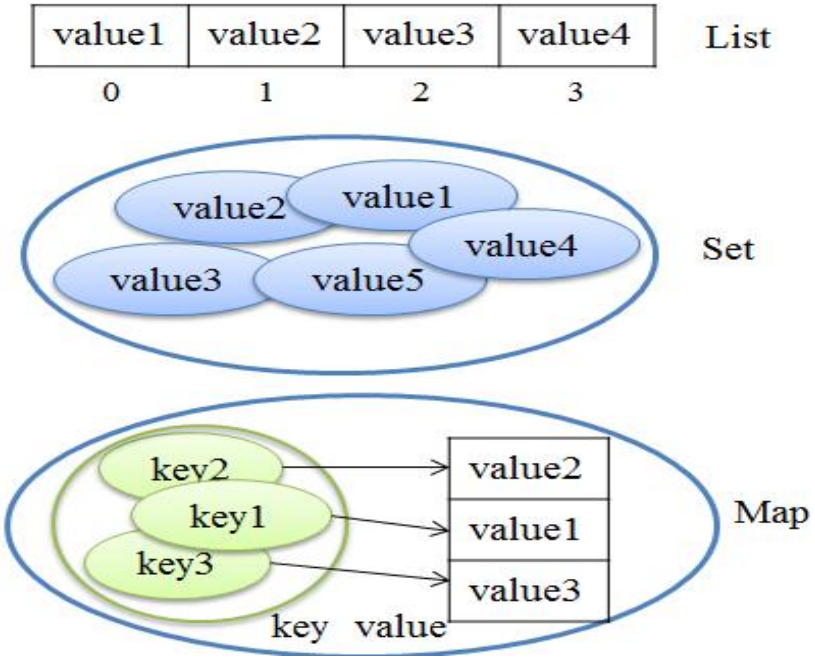
或者：

```
Set<Student> students = new TreeSet<>(new ComparatorNameAge());
```

4. HashSet和TreeSet的选用

- 原则：取决于集合中存放的对象，如果不需要对对象进行排序，那么就没有理由在排序上花费不必要的开销，使用HashSet即可。
- 散列的规则通常更容易定义，只需要打散排列各个对象就行。而TreeSet要求任何两个对象都必须具有可比性，可是在有的应用中比较的规则会很难定义。

- Map用于保存具有映射关系的数据，它们以键值对<key,value>的形式存在，key与value之间存在一对一的关系，多组键值对信息存放于Map集合中。Map集合将键、值分别存放，键的集合用Set存储，不允许重复、无序；值的集合用List存储，与Set对应、可以重复、有序。



8.5.1 Map接口

- Map接口中的常用方法：查看API文档

事实上，Java是先实现了Map，然后通过包装一个所有value都为null的Map实现了Set，在底层只有Map。

【例8-8】假设有一份学生名单，键是学生的id，值是Student对象（包括name和age信息）。建立HashMap对学生信息进行管理。

- HashMap的键在使用时，需要遵守与HashSet一样的规则：如果需要重写equals()方法，那么同时重写hashCode()方法，并保证两个方法判断标准是一致的，因为HashMap的键集就是一个Set。

- **2. HashMap迭代的方法**
  - Map的迭代方法较List和Set稍微复杂些，因为它本身是不能迭代的（未实现Iterable接口，不能用迭代器访问）。但从Map出发可以得到三个集合，即键集合、值集合、以及键值对集合，它们都可以被迭代。

（1）按键集合迭代

- 使用keySet()方法可以从Map获取键集，因为键集是Set，所以可以使用迭代器。在迭代过程中，使用get()方法从Map中按key获取到value。

```
Set keys = map.keySet();    //(1)取出map中的键集
Iterator it = keys.iterator();
while(it.hasNext()){
        String key = (String)it.next();
        Student stu = (Student)map.get(key);  //(2)按键从map中获取对应的值
        System.out.println(key+":"+"("+stu.getName()+","+stu.getAge()+")");
}
```

（2）迭代键值对集合

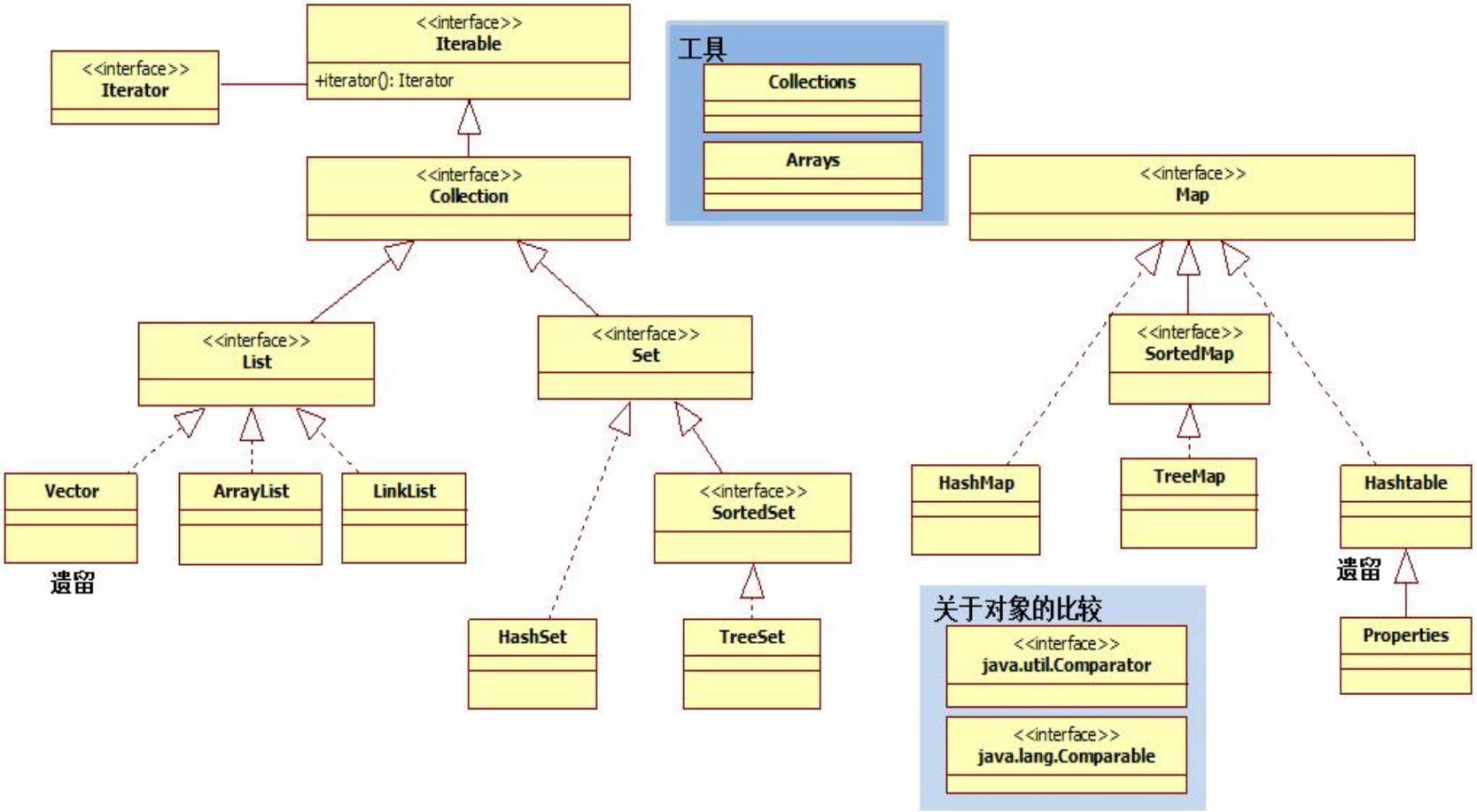
- Map内部定义了一个Entry类，它封装了一个键值对。
- 使用Map的entrySet()方法可以获取键值对集合。

- Entry中有3个方法，对每个键值对进行操作：
  - Object getKey()：返回Entry对象的key值。
  - Object getValue()：返回Entry对象的value值。
  - setValue(Object value)：设置Entry对象的value值，并将新值返回。

（2）迭代键值对集合

```
Set entrySet = map.entrySet();          //从map获取键值对集合
Iterator it = entrySet.iterator();
while(it.hasNext()){
        Entry entry = (Entry)it.next();          //获取一个键值对对象
        String key = (String)entry.getKey(); //从键值对中获取key
        Student stu = (Student)entry.getValue();        //从键值对中获取value
        System.out.println(key+":"+"("+stu.getName()+","+stu.getAge()+")");
}
```

8.6 回首Java集合框架

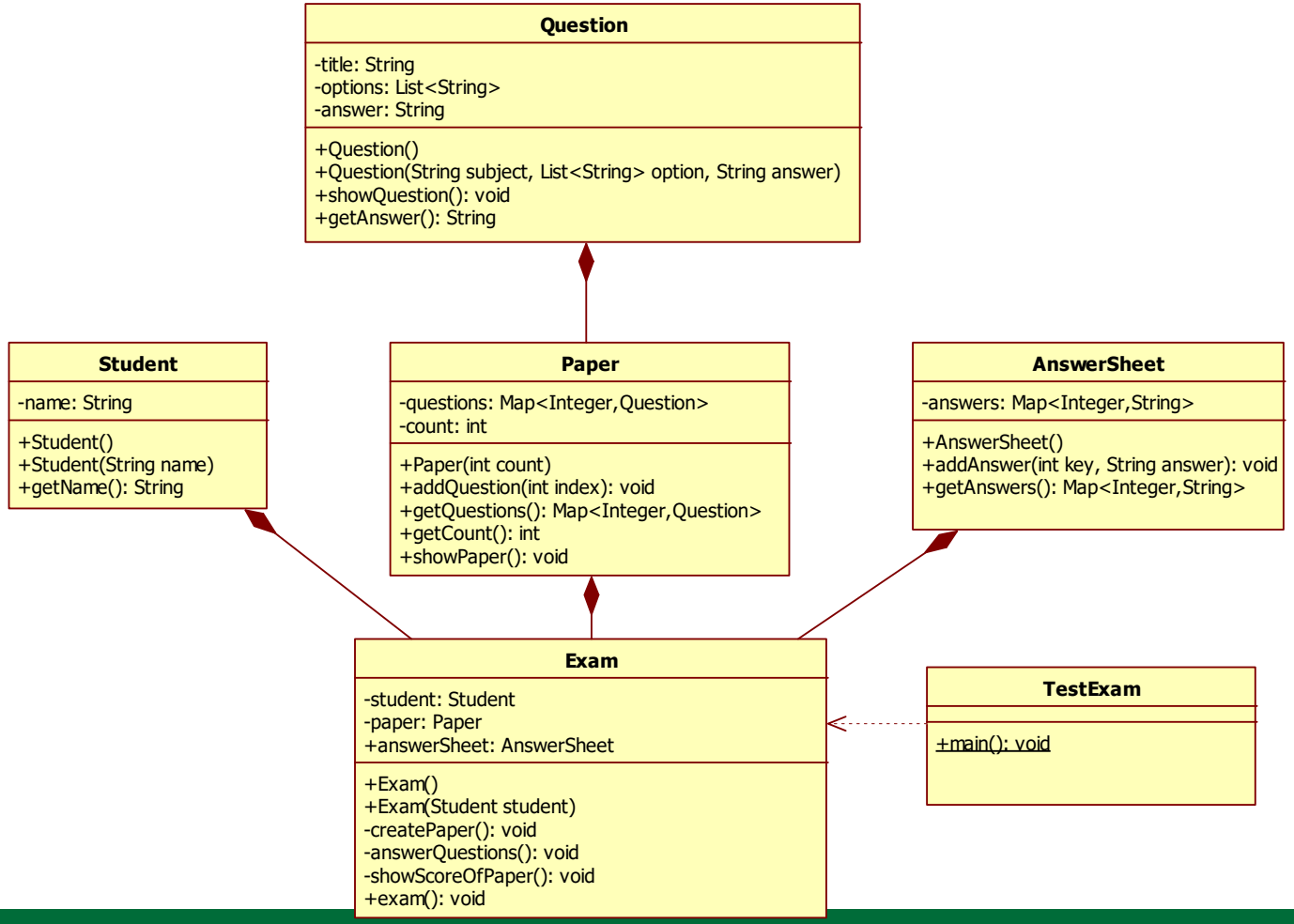


8.6 回首Java集合框架

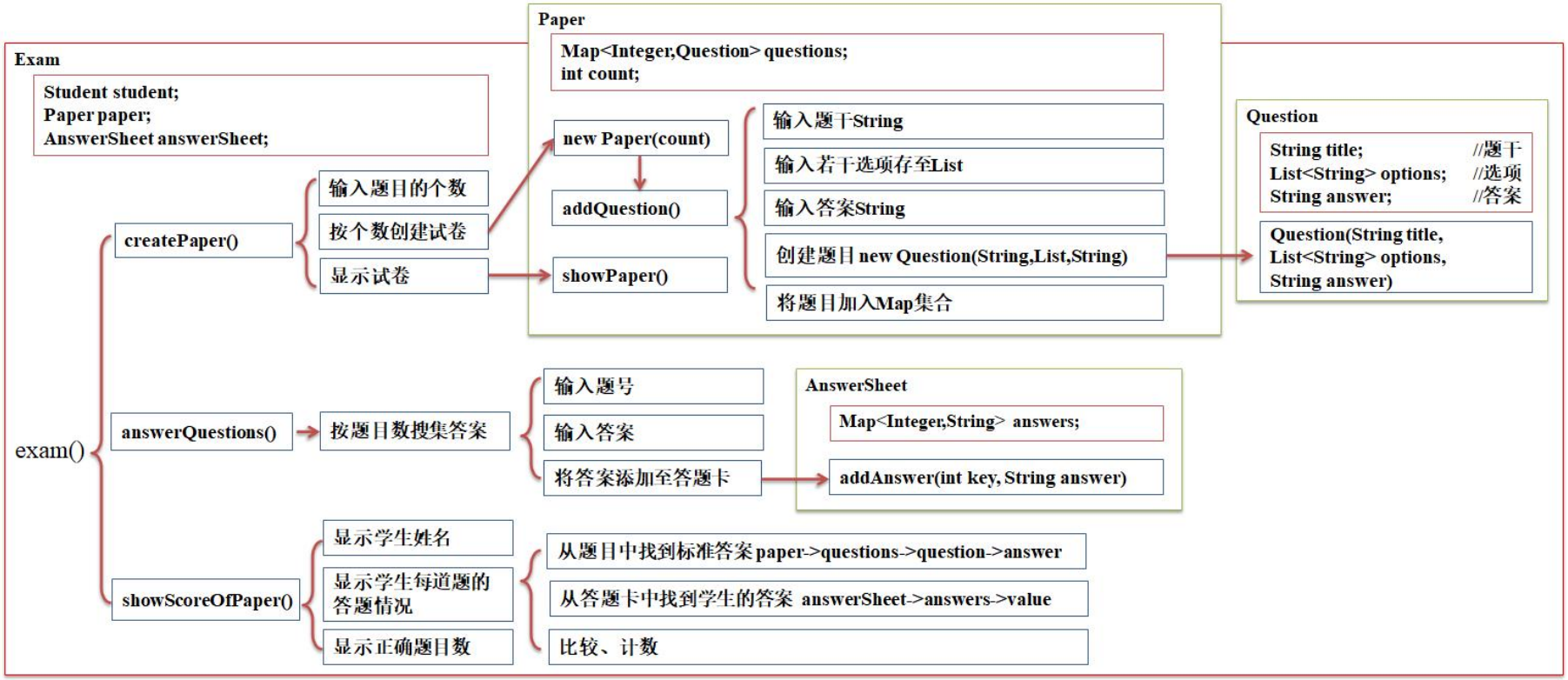
北京理工大学 BEIJING INSTITUTE OF TECHNOLOGY

接口	实现类	存储方式	优点	缺点	其他
List 有序	ArrayList	数组	按位置索引存取快	插入、删除、查找慢	
	LinkedList	双链表	插入、删除快	按位置存取、查找慢	
Set 无序	HashSet	散列表	插入、删除、查找快		需重写equals()和hashCode()方法
	TreeSet	二叉排序树	插入、删除、查找快 有序排列	速度慢于HashSet	需制定比较规则
Map 映像	HashMap	散列表	插入、删除、查找快	迭代效率较低	键需重写equals()和hashCode()方法
	Properties	散列表	读取属性文件便利		键、值均为String
	TreeMap	二叉排序树	插入、删除、查找快 键值有序排列	速度慢于HashMap 迭代效率低	键需制定比较规则

8.7 综合实践---控制台版考试系统



8.7 综合实践---控制台版考试系统



作业2：客观题

数组和集合的区别是什么？分别适用于什么场景？